Graph-Grounded Agent Prompting for Explainable Coordination-Risk Detection in Software Projects

Sanjaykumar N. B.
Department of Artificial Intelligence and Data Science
Saveetha Engineering College, Chennai, Tamil Nadu, India
sanjayynb@gmail.com

***Abstract*—Project-management tools report what a team has done; they rarely say what is about to go wrong, and when they do, the warning arrives without evidence a manager can check. The obvious remedy—hand the project to a large language model and ask what is at risk—costs tokens in proportion to project size, returns a different answer each time, and justifies itself with references that may not exist. This paper reports a system built on the opposite arrangement. Project state is compiled into a typed knowledge graph, six categories of coordination risk are computed from that graph by deterministic algorithms, and the language model is given only a bounded witness subgraph around one already-detected anomaly, with the single job of narrating it. Every citation the model produces is resolved against the node identifiers in that subgraph and discarded if it does not resolve. Across a project grown from 13 to 2,003 tasks the prompt grew 1.47× while full-dataset prompting grew to 189,834 tokens, a 487× reduction at the largest size; 200 of 200 deliberately fabricated citations were removed; and agreement between the model's narrated verdict and the deterministic one reached Cohen's $\kappa = 0.95$. On 270 sprints from 22 previously unseen open-source projects, the same graph projected forward flags late sprints at the halfway mark with AUC 0.792, rising to 0.874 at three-quarters. We also report what did not work: a tuned logistic model reached cross-validated F1 0.816 and fell to 0.593 on unseen projects, where the zero-parameter graph rule scored 0.712, and the mid-sprint projection ranks no better than counting unfinished work.**

***Index Terms*—Knowledge graphs, large language models, multi-agent systems, explainable artificial intelligence, software project management, risk prediction, mining software repositories.**

I. Introduction

A sprint that finishes late rarely fails for a reason nobody could see. The dependency that blocked everything was in the issue tracker. The engineer carrying three times the team's median load was visible on the board. The component only one person had ever touched was a matter of record. What was missing was not data but a reading of it, early enough and specific enough to act on.

Commercial trackers—Jira, Linear, Asana—are built to record state, and they do it well. Where they offer risk signals at all, two properties limit their usefulness. The signal arrives after the fact, computed from a snapshot rather than projected forward; and it arrives without justification, so a manager can neither act on it with confidence nor argue with it. A flag that says *at risk* and nothing more transfers no information a team can use.

The arrival of capable language models suggested a shortcut: serialise the project, ask the model what is going wrong, and read the answer. Having built that baseline and measured it, we think the shortcut is a poor trade for three separate reasons. Cost scales with the project, because the whole board has to enter the context window; a 2,000-task project costs roughly 190,000 prompt tokens per analysis, which rules out running the analysis on every change. Determinism is lost, because the same board can yield different verdicts on consecutive calls, and a risk score that moves when nothing moved is not a measurement. Finally, and most seriously for a tool whose output is meant to change what people do, the justification cannot be trusted: models cite issue keys and names that look exactly like real ones and are not [12].

This paper describes a system, TeamSync AI, built on the inversion of that arrangement. Risk is *computed*, not inferred. Project state—people, tasks, milestones, dependencies, comments—is compiled into a typed knowledge graph, and each of six risk categories is expressed as a property of that graph: the longest path through the dependency DAG, articulation points separating a person from a component, weighted degree over assignments, the ratio of commenting to assignment, and so on. These are ordinary graph computations [2], they take milliseconds, and they give the same answer twice.

The language model is not removed; it is confined. Once a risk has been computed, a *witness subgraph* is extracted around the nodes responsible for it—one hop, at most eight neighbours per seed, at most sixty nodes—and that subgraph, and nothing else, is placed in the prompt. The model's task is to explain the finding in a sentence a manager can read, citing node identifiers. Afterwards, every identifier it cited is looked up in the subgraph it was given, and anything that fails to resolve is deleted before the output is shown. The model never sees the project, never computes the score, and cannot invent an issue key that survives to the user interface.

Three properties follow from that confinement, and each is measured in Section VII. Prompt size stops tracking project size, because the subgraph is bounded by construction. Hallucinated evidence becomes mechanically detectable rather than a matter of trust, because the set of legitimate references is finite and known. And the system degrades cleanly: every agent has a rule-based path producing the identical output schema, so with no API key configured the product still works, which is also how every number in this paper was measured.

The contributions are:

- an architecture in which coordination risk is a deterministic graph computation and the language model is a bounded narrator, with one explicitly drawn interface between them (Sections III–IV);

- a witness-subgraph construction whose prompt cost is a function of the number of anomalies rather than of project size—measured at 1.47× prompt growth across a 154× growth in project size, and a 487× reduction against full-dataset prompting at 2,003 tasks;
- post-hoc mechanical grounding of citations against a bounded identifier set, removing 200 of 200 injected fabrications;
- an evaluation on 458,232 real Jira issues with a project-level held-out split, reporting mid-sprint early warning at AUC 0.792 and 0.874; and
- three negative results, reported in full, including a tuned model whose advantage did not survive unseen projects and a projection that ranks no better than a trivial baseline.

II. Related Work

A. Risk and effort prediction in agile projects

Predicting whether agile work will land on time is an old problem with a sobering literature. Jørgensen and Shepperd's review of cost-estimation research [17] and Hall et al.'s review of fault-prediction performance [16] both report that reported accuracy depends heavily on evaluation design, and that results rarely transfer between contexts. Deep learning has been applied directly to story-point estimation, most prominently by Choetkiertikul et al. [18], whose Deep-SE model learns effort from issue text. A subsequent replication by Tawosi et al. [19] found that Deep-SE beat a median baseline with statistical significance in only a small minority of cases. We read that pair of papers as a caution rather than a discouragement, and it is one reason the present work reports a learned model that lost on held-out data instead of quietly dropping it.

B. Multi-agent systems built on language models

A fast-moving body of work composes language models into cooperating agents. MetaGPT [4] assigns standard-operating-procedure roles to agents producing software artefacts; ChatDev [5] runs a simulated software company through conversational turn-taking; AutoGen [20] generalises the pattern into a conversation framework; ReAct [7] interleaves reasoning traces with tool actions; and AgentBench [6] and the survey of Wang et al. [21] map the resulting design space. These systems share an assumption worth naming: that the model is the component that *decides*, and that the engineering problem is arranging its decisions well. The present work takes the opposite position for the specific task of risk detection—the decision is a graph computation, and the model is kept out of it—so the comparison is architectural rather than a matter of better prompting.

C. Explainability

Post-hoc explanation methods such as LIME [9] and SHAP [8], surveyed broadly by Guidotti et al. [22] and Arrieta et al. [23], answer the question *why did this model predict that?*, and Jiarpakdee et al. [10] apply them specifically to defect-prediction models in software engineering. They are the right tools when a learned model has made the prediction. Here they are not applicable, and the reason is structural rather than a claim of superiority: a critical-path share or an articulation point has no hidden decision to reconstruct. The explanation is the computation. What remains to be explained is only the *prose*, and the risk there is fabrication rather than opacity—which is why the mechanism in Section IV-E is a verifier and not an attribution method.

D. Grounding generated text

Retrieval-augmented generation [11] conditions a model on retrieved passages so that its output has a documentary basis, and the surrounding literature on hallucination [12] establishes how often that basis is still embellished. Pan et al. [24] survey the broader union of language models and knowledge graphs. The distinction here is one of direction and of timing: retrieval conditions generation *beforehand* on text, while the present system verifies generation *afterwards* against a computed, typed structure whose legitimate identifiers form a closed set. Conditioning makes fabrication less likely; closed-set verification makes it detectable. The two are complementary, and the second is what this paper measures.

E. Where this work sits

No comparator we are aware of combines all five of: specialist multi-agent decomposition; risk computed by graph algorithm rather than inferred by a model; mechanically enforced evidence grounding; a deterministic fallback emitting an identical schema; and a held-out real-world evaluation that reports its negative results.

III. System Architecture

The deployed system is three tiers plus two stores, shown in Fig. 1. A Next.js client presents a Kanban board, a risk dashboard and project settings. A FastAPI application tier handles authentication, organisation-level tenancy, work items and repository synchronisation, and owns the system of record in PostgreSQL. A second FastAPI service performs analysis; it holds the graph machinery and is the only component that may talk to a language model.

The split between the two services is deliberate and is not merely about deployment convenience. The application tier is where data lives and where authorisation is enforced; the analysis tier is stateless, receives a project snapshot over an internal interface, and returns findings. A snapshot in, findings out means the analysis service can be run, tested and reasoned about without a database, and it is why the entire evaluation harness in Section VI can replay 987 historical sprints through exactly the code path the product uses.

Within the analysis tier the figure draws the distinction the paper is about. The deterministic core compiles the graph, computes the risk metrics and selects the witness subgraph; it performs no model call and contains no randomness. The narration layer assembles a prompt, calls a model adapter and verifies the result. The arrow between them carries the witness subgraph and nothing else. The dashed line leaving the core shows the risk scores travelling back to the application tier directly: if the narration layer fails, is rate-limited, or is switched off entirely, the numbers the product displays are unchanged.

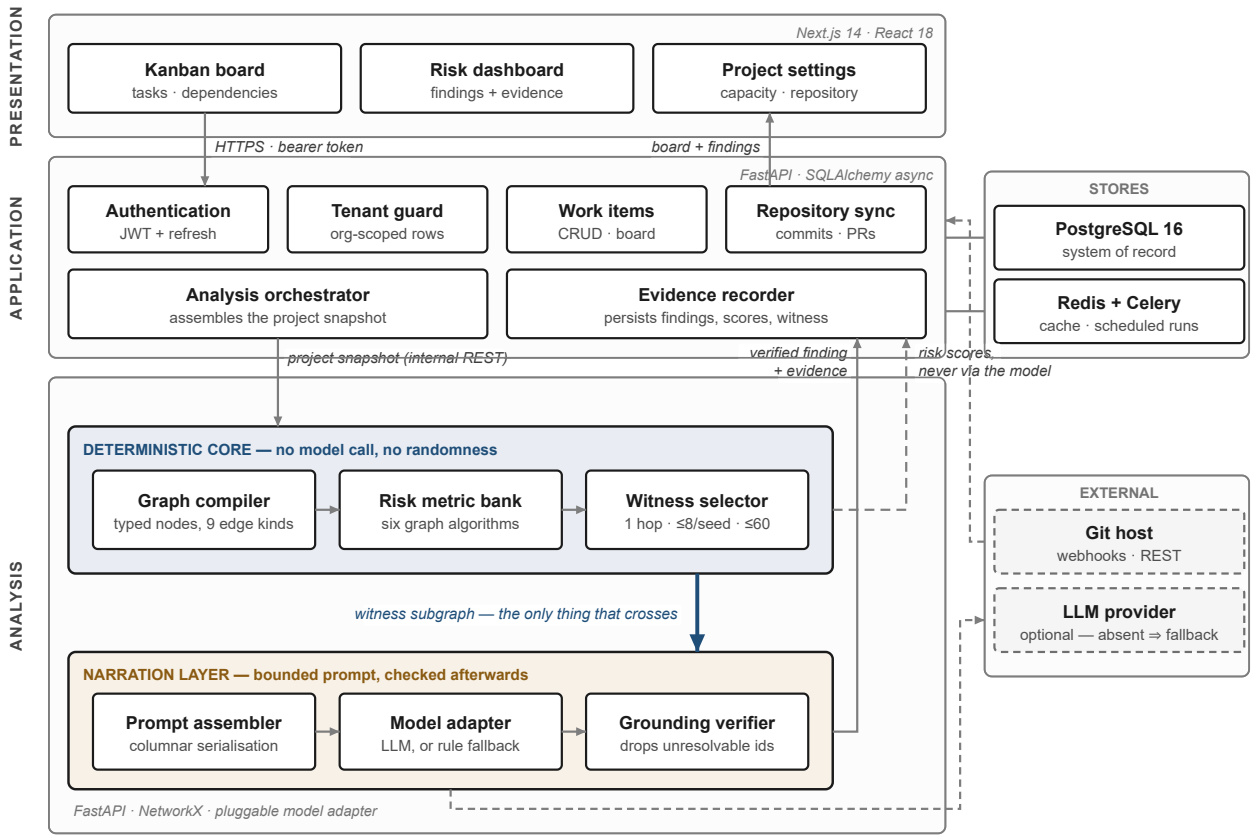


Fig. 1. System architecture. Blocks are deployed subsystems; lines are interfaces between them. The analysis tier is divided into a deterministic core, which performs every computation that produces a number, and a narration layer, which may call a language model. Exactly one artefact crosses the boundary between them—the witness subgraph—and the risk scores reach the application tier without passing through the narration layer at all (dashed). The external LLM provider is optional: with no key configured the model adapter uses a rule-based path that emits the same schema, which is the configuration under which every result in Section VII was measured.

IV. Graph-Grounded Prompting

A. The project graph

A project snapshot is compiled into a directed multigraph over five node types—`Person`, `Task`, `Milestone`, `Project` and `Component`—connected by nine edge types. Assignment edges carry story points; dependency edges come from issue links; commenting edges come from the comment table; and one derived edge, `Person KNOWS Component`, is the composition of assignment with component membership, which is what makes single-point-of-failure analysis possible without asking anybody to maintain a skills matrix.

Every edge is derived from tables the product already keeps. No new instrumentation is introduced, nobody is asked to fill in a form, and the graph for a project of a few thousand tasks is built in single-digit milliseconds. That matters for a reason beyond efficiency: a risk analysis that requires its own data collection does not get run.

B. Risk as a graph property

Each of six risk categories is a question about the graph, with a documented threshold (Table I). None of them involves a model, and none of them involves a fitted parameter. Building the graph also produced a correctness result that was not sought: the pre-existing API rejected a task that depended on itself but accepted a three-hop cycle A → B → C → A, because it only checked for direct self-loops. Once the graph exists, genuine cycle detection is a library call.

TABLE I
The six risk categories as graph computations

Risk	Computation	Threshold
Dependency	Longest path through the `BLOCKS` DAG	≥ 0.25
Concentration	Betweenness centrality [25]	—
Knowledge	Articulation points, Person–Component	≥ 2 tasks
Workload	Weighted degree over assignments	≥ 1.50
Coordination	Community structure [26], isolated members	—
Silent member	Comment degree ÷ assignment degree	≥ 0.30
Delay	Open tasks past due date	≥ 0.10

C. The witness subgraph

When a metric fires, the nodes responsible for it become seeds, and the subgraph induced by their immediate neighbourhood becomes the witness. Three parameters bound it: one hop; at most eight neighbours per seed; at most sixty nodes in total, with seeds guaranteed to survive truncation.

The middle parameter is the one that matters, and we only learned that by getting it wrong. An earlier implementation

expanded every neighbour of every seed. Because real project graphs have hubs—a release milestone touching four hundred tasks, a tech lead assigned to a third of the board—a single high-degree seed dragged a large fraction of the project back into the prompt, and measured prompt growth was 4.31× across the size sweep. Capping neighbours per seed brought it to 1.007×. The bound on prompt size is not a property of taking one hop; it is a property of refusing to follow a hub.

Serialisation is columnar rather than JSON. Field names appear once in a block header, for instance `tasks(id,status,pts,due,title):`, followed by rows. For a sixty-node witness this removes several hundred repeated key tokens, which is a mundane saving next to the subgraph bound but is free.

D. What the model is asked to do

The prompt contains one finding, its witness, and an instruction to explain the finding in prose, citing node identifiers from the witness. It does not contain the project, the other five risk categories, or the score. The model is not asked whether the finding is correct, and its answer is not used to compute anything. Section VII-F reports how closely its narrated severity tracks the deterministic one; the point of that measurement is to confirm the model is narrating rather than quietly re-deciding.

E. Verifying the citations

After generation, each cited identifier is resolved against the identifiers present in the witness. Unresolvable citations are dropped, and if dropping them empties the evidence list the finding is still shown—with its computed score and no narration—rather than being suppressed. This is a closed-set membership test, not a plausibility judgement, which is the whole reason it can be stated as a guarantee rather than a tendency.

One implementation detail is worth recording because it was a silent failure for some time. The verifier treated an empty list of legitimate identifiers as *no filter configured* rather than as *nothing is legitimate*, so in exactly the situation where every citation should have been rejected, every citation passed. The test that caught it injects fabricated references deliberately; the result in Section VII-C is from that test.

V. Multi-Agent Design

Five agents are active. A coordinator assembles the snapshot and runs three specialists concurrently—Planning, which reads milestones and capacity; Progress, which reads velocity and stalled work; and Workload, which reads assignments and single-point-of-failure structure. A risk aggregator consumes the six computed scores, and a recommendation agent turns the aggregate into prioritised actions. Two further agents, covering meeting transcripts and chat traffic, are specified but not implemented, for the plain reason that the product has no such data source; they are listed here because omitting them from a system diagram would overstate what was built.

Every agent, on both the model path and the fallback path, emits the same structure: a summary, a severity level, a confidence, weighted signals, evidence items, recommendations, and a metadata field that records which path produced it. The uniformity is what makes the fallback honest. A caller cannot tell from the schema whether a model was involved, so no downstream code is written to depend on one, and the deterministic path is not a degraded mode bolted on afterwards—it is the reference implementation, and the configuration in which this paper's results were produced.

VI. Experimental Setup

A. Dataset

Real-world evaluation uses TAWOS [1], a published collection of agile open-source projects drawn from twelve public Jira installations. We parsed the full dump with no parse errors and reconciled the row counts against the published totals: 39 projects, 206,162 users, 4,594 sprints, 458,232 issues, 246,587 issue links and 1,518,327 comments.

TAWOS has no due-date field, so the prediction target must be derived. A sprint is labelled *delayed* when at least 30% of its issues were unresolved at sprint end. This is defensible and it is a proxy; Section IX treats it as the first threat to validity rather than a settled definition. Restricting to closed sprints with an end date and at least fifteen issues gives 987 eligible sprints across 31 projects, 58.8% of them positive.

B. Point-in-time reconstruction

Every issue in an archival dataset has long since been resolved. Using current status would leak the future into the prediction and produce excellent, meaningless numbers. Each sprint is therefore rebuilt as it stood at the moment of evaluation: only issues created by then exist, only issues resolved by then are complete, and the system clock is set to that moment. For the sprint-end evaluation the clock is the deadline plus one day; for the mid-sprint evaluation it is the 50% and 75% points of the sprint's duration.

An early version of this harness set the due date equal to the evaluation time, so the condition `due < now` was never true and recall was 0.0 on the first run. The bug is mentioned because its symptom—a detector that fires on nothing—is much easier to notice than the opposite, and the opposite is what a leaking harness produces.

C. Splitting

Projects, not sprints, are the unit of splitting. Sprints inside one project share a team, a codebase and a set of Jira conventions; splitting by sprint would let those conventions appear on both sides of the boundary. Nine projects and 717 sprints form the tuning set; 22 projects and 270 sprints (48.9% positive) form the held-out set, which was evaluated exactly once. Checkpoints, predictor and severity cut-offs were all fixed before the held-out projects were touched.

D. Synthetic scenarios

Real data exercises one of the six risk categories. To check the implementation of the other five, 180 scenarios—90 positive and 90 negative, 30 per category—are generated with a deterministically injected anomaly, so ground truth holds by construction. These support correctness checking, a boundary sweep and the ablation, and Section VII-B states plainly what they do and do not demonstrate.

E. Metrics

Detection quality is reported as precision, recall and F1, with accuracy included but de-emphasised given the class balance;

AUC-ROC for threshold-independent ranking; and the Brier score for calibration. Paired comparisons use exact McNemar tests on the same sprints, and interval estimates use a cluster bootstrap that resamples *projects* with replacement over 2,000 replicates [3], matching the unit of the split. Model–rule agreement is Cohen's κ [14], read on the Landis–Koch scale [15]. Prompt cost is reported in tokens, and latency as p50 and p95.

VII. Results

A. Prompt cost does not follow project size

Table II and Fig. 2 give the efficiency result, measured identically for both architectures. As the project grows from 13 to 2,003 tasks—a factor of 154—the graph-grounded prompt grows from 266 to 390 tokens, a factor of 1.47. Naive full-dataset prompting grows from 1,050 to 189,834 tokens. At the largest size the ratio is 486.8×, a 99.8% reduction.

The shape of the curve matters more than the ratio. The graph-grounded series is effectively flat after a few hundred tasks, because the witness is bounded; the naive series is linear, because the board is the prompt. The practical consequence is a change in what the system can afford to do. Deterministic analysis of a 203-task project takes 10.96 ms at the median and 13.42 ms at p95 over 50 runs, which is fast enough to run on every board change, synchronously, without a job queue. Analysis that costs 190,000 tokens is something a team runs nightly, which returns us to detecting risk after it has happened.

TABLE II
Prompt tokens against project size

Tasks	Graph-grounded	Naive	Ratio
13	266	1,050	3.9×
53	386	4,695	12.2×
203	389	18,901	48.6×
1,003	389	94,724	243.5×
2,003	**390**	**189,834**	**486.8×**

B. Implementation correctness on synthetic scenarios

On the 180 synthetic scenarios the detectors score 90 true positives, 90 true negatives and no errors of either kind, for precision, recall, F1 and accuracy of 1.00, identically across four further seeds and 1,200 additional trials.

This number should be read for what it is. Deterministic threshold code evaluated on inputs constructed to be unambiguous is *expected* to score perfectly; the result is equivalent to a passing test suite, and it is evidence of correct implementation, not of generalisation. The genuinely falsifiable check in this family is the boundary sweep, which asks whether each detector changes its verdict exactly where its documentation says it does. The overdue-ratio detector flips exactly at 0.10 and the silent-member detector exactly at 0.30; the workload detector's flip is bracketed between 1.429 and 1.522 around its documented 1.50, which is the resolution the sweep can establish.

C. Grounding

Across a test corpus in which fabricated references were injected deliberately, 200 of 200 were removed, leaving no unresolvable citation in any output. The guarantee is narrow and should be stated narrowly: every surviving citation names a node that exists in the witness the model was shown. It does not follow that the surviving citation is the *most relevant* node, and nothing here measures whether the resulting explanation helps a manager—see Section IX.

D. Mid-sprint early warning on unseen projects

This is the result that answers a question a project manager actually has, at a time when the answer can still change what they do. At each checkpoint the sprint is reconstructed as it then stood, the share of scope still unfinished at the deadline is projected from work completed against time elapsed, and delay risk is raised using the same severity cut-offs the product uses everywhere else. Nothing was tuned for this experiment. Table III reports the 270 held-out sprints from 22 unseen projects.

TABLE III
Mid-sprint early warning, 270 held-out sprints from 22 unseen projects (48.9% positive)

Checkpoint	Predictor	Precision	Recall	F1	Flag rate	AUC	ΔF1 vs. flag-all (95% CI)	McNemar
50%	Delay risk ≠ low	0.581	0.902	**0.706**	0.759	**0.792**	+0.050 [+0.006, +0.094]	52 vs. 13, $p < 0.001$
50%	Flag every sprint	0.489	1.000	0.657	1.000	—	—	—
50%	Open-work-share baseline	—	—	—	—	0.802	—	—
75%	Delay risk ≠ low	0.623	0.962	0.756	0.756	**0.874**	+0.099 [+0.049, +0.148]	61 vs. 5, $p < 0.001$
75%	Delay risk high or worse	0.719	0.909	**0.803**	0.619	—	—	—
75%	Flag every sprint	0.489	1.000	0.657	1.000	—	—	—
75%	Open-work-share baseline	—	—	—	—	0.881	—	—

Halfway through a sprint the warning carries information: it drops roughly a quarter of the sprints that a flag-everything policy would raise, while still catching 90% of those that finish late, and the paired tests separate it from that policy at both checkpoints. By three-quarters the separation is clear, with AUC 0.874.

What the table also shows, and we would rather say it than have a reviewer find it, is that the graph projection does not *rank* better than a trivial alternative. Simply measuring the share of work still open gives AUC 0.802 at the halfway point against the projection's 0.792, and 0.881 against 0.874 at three-quarters. The projection earns its place in the system by being the same object that explains the risk and carries its evidence, not by ordering sprints more accurately than counting unfinished issues.

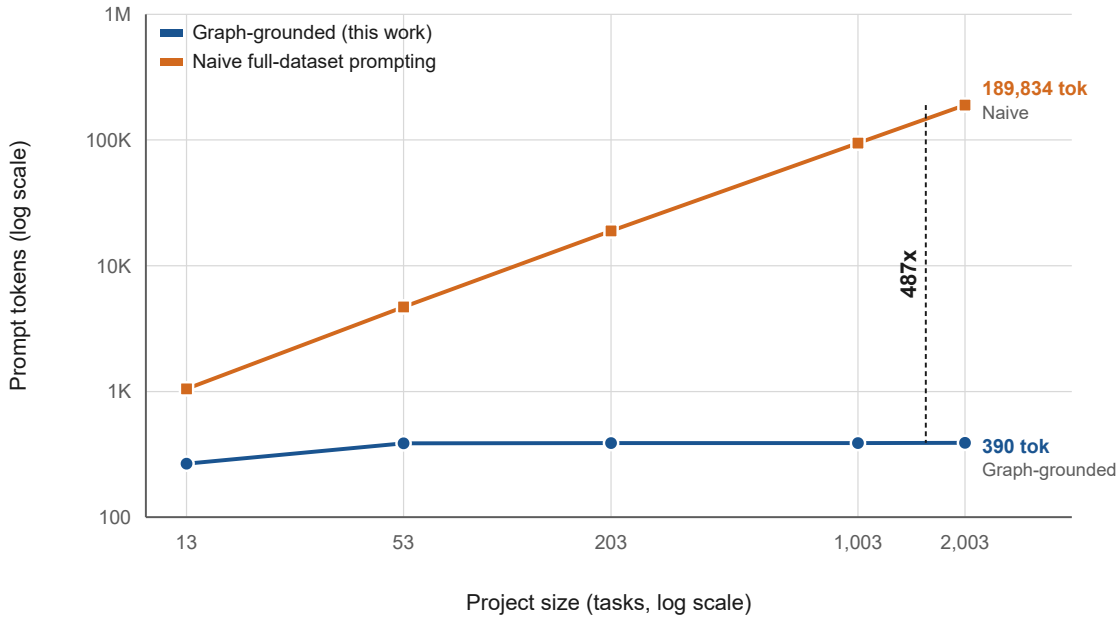


Fig. 2. Prompt tokens against project size, both axes logarithmic. The graph-grounded series flattens once the witness bound binds; the full-dataset series grows linearly with the board. The annotated gap at 2,003 tasks is 486.8×.

E. Sprint-end evaluation, and a learned comparator

Evaluated after the deadline on the same held-out projects (Table IV), the zero-parameter graph rule reaches F1 0.712 with recall 1.000, and a tuned four-feature logistic model reaches 0.593. The cluster bootstrap puts the rule's F1 interval at [0.610, 0.814]. Fig. 3 sets both against the story-point baseline.

TABLE IV
Sprint-end, 270 held-out sprints

Model	P	R	F1	Acc	AUC
Graph rule (0 parameters)	0.552	1.000	**0.712**	0.604	—
Tuned logistic composite	0.620	0.568	0.593	0.619	0.658
Flag every sprint	0.489	1.000	0.657	0.489	—

Two cautions belong with these numbers. The recall of 1.000 is *structural*: after the deadline, a delayed sprint is by definition one with unfinished work past its due date, which is exactly the condition the rule tests. It is a property of evaluating after the fact, not a skill, and its bootstrap interval of [1.000, 1.000] is correct precisely because the quantity has no variance—not one of the 22 held-out projects produces a false negative. The mid-sprint evaluation in Section VII-D, where the outcome is not yet determined, is where recall means something.

The second caution concerns what may be concluded about the learned model. The rule's F1 exceeds the composite's point estimate, but a paired test cannot separate them: exact McNemar gives $p = 0.78$, and the bootstrap difference is +0.119 with a 95% interval of [−0.025, +0.291] spanning zero. Against flagging every sprint, by contrast, the difference is +0.055, [+0.030, +0.080], $p < 0.001$—a real effect. The defensible claim is therefore not that the rule predicts better than the learned model. It is that the learned model's tuned advantage did not survive the move to unseen projects, and that the simpler object is the one that arrives with its evidence attached.

F. Three attempts that failed

Reporting these is not modesty; they are the evidence that the simple rule is not leaving accuracy unclaimed.

1) The threshold sweep was inert. Sweeping the overdue threshold from 0.05 to 0.50 produced byte-identical output at every value. The reason is structural: with a single shared sprint-end deadline, the overdue ratio is always exactly 0.0 or 1.0, so no threshold strictly between them can reclassify anything. A sweep that cannot change its output is not a weak result, it is a misconceived experiment, and it was discarded as such.

2) Individualised due dates made ranking worse. Deriving per-issue deadlines from observed cycle time (2.96 days per story point) did un-collapse the signal, moving the median overdue ratio to 0.76, but AUC fell from 0.581 to 0.531. Story points are too noisy a duration proxy to carry per-issue deadlines—a finding consistent with the replication literature on story-point estimation [19].

3) The learned model overfitted across project boundaries. A four-feature logistic model with three-fold group cross-validation by project reached CV F1 0.816 and AUC 0.887, and fell to F1 0.593 and AUC 0.658 held out. Its dominant weight landed on an idle-member ratio that is mechanically coupled to the label rather than independently predictive. An AUC collapse of that size across a project boundary is the signature of fitting project-specific structure, and it is the reason this paper leads with a rule that has nothing to fit.

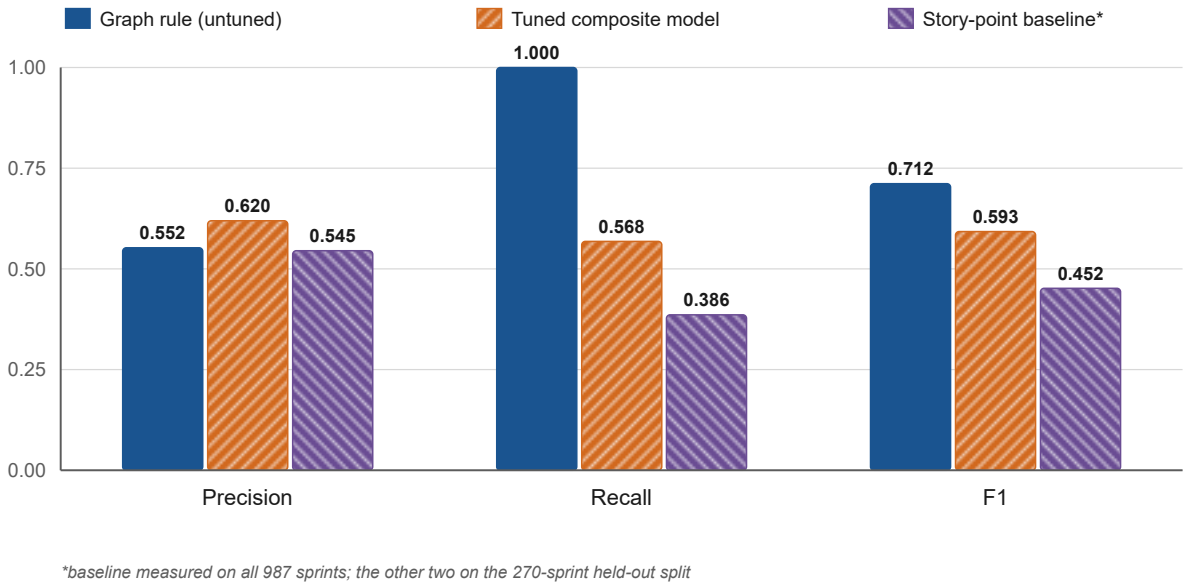


Fig. 3. Held-out performance on 270 sprints from 22 unseen projects. Bars carry hatch patterns and direct value labels because IEEE pages are printed in greyscale, where the fill colours would otherwise collapse.

G. Does grounding cost accuracy?

The ablation scores both architectures on identical scenarios against identical ground truth (36 scenarios, 18 positive, no failed calls). The graph-grounded path scores F1 1.000; naive full-dataset prompting scores 0.947, with per-category results identical on five of six (Fig. 4). The entire gap is coordination risk, where the naive path reaches F1 0.75 at precision 0.60.

The comparison is deliberately tilted toward the naive baseline: tiny projects where full context is cheap and fits comfortably, a good-faith prompt defining all six categories, and lenient response parsing. Three qualifications still apply. The naive baseline is not bad—F1 0.947 is a respectable score, and it over-reports only modestly, naming 2.75 risk categories per scenario against the graph's 2.42, with 12.1% of its claims uncorroborated. Scoring is per-target-category, so risks it reports on other dimensions are not counted against it; stricter scoring would likely widen the gap, and we do not claim it. And nothing here says anything about naive accuracy *at scale*, which is untested and, at 190,000 tokens per scenario, largely untestable on an ordinary budget—which is itself part of the argument. The mean prompts in this ablation were 1,109 tokens against 1,804, a ratio of 1.6×; the 487× figure belongs to Section VII-A and the two should not be conflated.

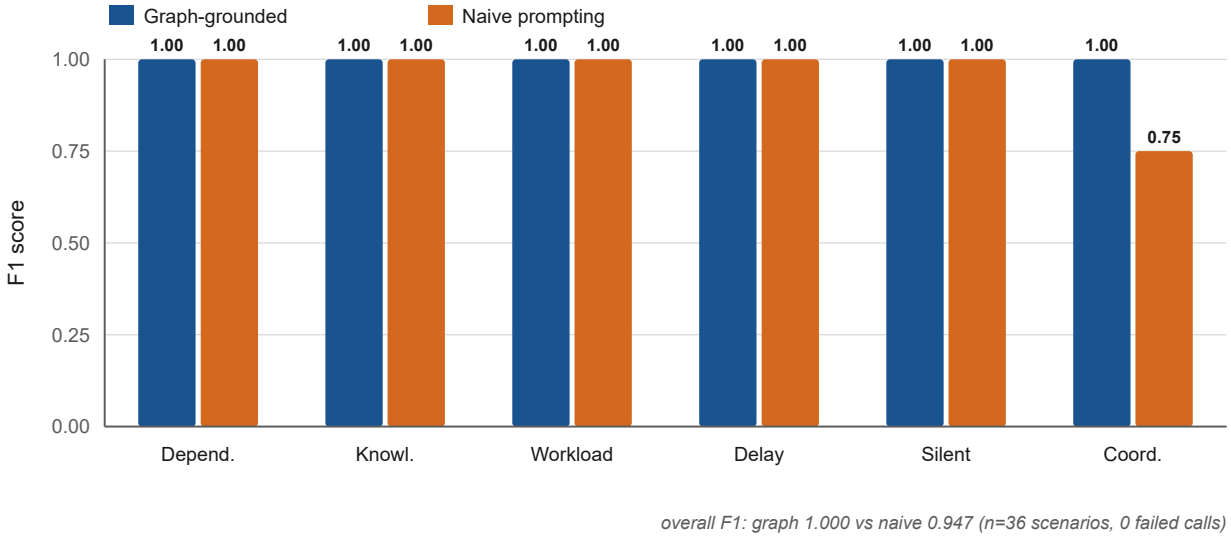


Fig. 4. Per-category F1, graph-grounded against naive full-dataset prompting, on identical scenarios. Five categories are identical; the whole difference is coordination risk.

H. Does the model narrate, or quietly re-decide?

The grounding guarantee would be hollow if the model were silently overriding the computed verdict. To test that, 90 live calls were made across all six categories, and the narrated severity was compared with the deterministic verdict on identical input. Cohen's $\kappa = 0.9533$ over the 37 schema-valid responses—almost perfect agreement on the Landis–Koch scale [15]—with per-category agreement of 1.00 for dependency ($n = 14$) and workload ($n = 8$), and 0.86 for knowledge ($n = 12$).

Schema validity needs two numbers rather than one, and we report both. Over all 90 attempted calls, 41.1% returned a valid structure. Of the 24 individually diagnosed failures, 22 were HTTP 429 rate limits from a free-tier key—an infrastructure property, not a model property. Because the run logged only those 24 failures individually, validity over calls that actually reached the model cannot be computed, only bounded: between 54% and 95%. We quote 41.1% as the measured end-to-end rate and the range as a range. The commonly convenient figure of roughly 92% assumes the optimistic extreme, and this run has no per-call record that would support it.

I. Engineering baseline

The implementation is roughly 21,000 lines. Its test suites—109 backend tests on both SQLite and PostgreSQL 16, including a 185-check role-by-endpoint matrix; 85 analysis-service tests; 63 frontend tests—run on every push alongside a replay of the demonstration that re-checks every number the demonstration script quotes. Every figure is regenerated from committed result files, so a figure cannot drift away from the evaluation that produced it.

VIII. Discussion

The results support a narrower claim than the architecture might invite. What is demonstrated is that confining a language model to narration costs nothing measurable in detection quality, buys two orders of magnitude in prompt cost at realistic project sizes, and converts evidence fabrication from a hazard into a closed-set membership test. What is *not* demonstrated is that the graph projection is a better predictor of lateness than simple alternatives. On ranking it is not better than counting open work, and against a tuned learned model the comparison is statistically a draw.

We think that combination is the interesting finding rather than a disappointment. The field's default assumption is that a more sophisticated model buys accuracy, and that explainability is paid for out of that accuracy. Here the sophisticated model bought nothing that survived a project boundary, while the explainable object cost nothing and arrived with the graph nodes behind its verdict. When two predictors cannot be separated statistically, the tiebreakers are the ones a practitioner actually cares about: cost, determinism, and whether the output can be checked.

The operating point deserves comment. The rule is high-sensitivity and over-flags: roughly half its sprint-end alarms are on sprints that finish on time. That is the correct trade only when a false alarm is cheap to dismiss and a miss is expensive, which is the usual shape of sprint risk but is not universal. A team drowning in alerts will ignore all of them, and no confidence interval in this paper addresses that.

Finally, a system that scores people—identifying the overloaded engineer, the silent team member, the single point of failure—can be used against them. The design choices here reduce but do not eliminate the risk: the output is an explanation with its evidence rather than a bare score, which at least makes a claim about a person contestable by the person. We would not deploy this as an input to performance review, and we think a paper proposing it should say so.

IX. Threats to Validity

Construct. The delay label is a proxy: TAWOS has no due dates, and "at least 30% unresolved at sprint end" is a defensible choice rather than a canonical one. The sprint-end recall of 1.000 is guaranteed by that definition at that evaluation point and should not be read as predictive skill.

Internal. AUC is weak by construction at sprint level, because every task in a reconstructed sprint shares one deadline. The blocked-ratio signal never fires on this dataset, whose status vocabulary has no blocked state. At the 25% checkpoint the elapsed fraction falls below the projection's minimum and the signal is deliberately switched off, so nothing is claimed about the first quarter of a sprint.

External. Only one of the six risk categories—delay—is validated on real data. Single-point-of-failure, cycle and coordination signals are almost entirely absent from sprint reconstruction, because issue links in TAWOS span a project's whole history rather than one sprint; those five categories are validated synthetically, which tests implementation rather than generalisation. All data is open-source Jira, and company projects may differ.

Conclusion. The live-model sample is thin and imbalanced: $\kappa = 0.95$ rests on 37 observations with three of six categories at $n = 1$, from a single provider, a single model and one rate-limited key. Schema validity beyond the end-to-end rate is a bounded range, not a measurement. There is no human evaluation of explanation quality; grounding proves citations are real, not that explanations are useful. And there is no longitudinal study: no evidence is offered that a flagged risk, acted upon, changes an outcome weeks later. Two designed agents remain unimplemented.

X. Conclusion

Asking a language model to find risk in a project is expensive, non-reproducible and unverifiable. Computing the risk from a typed graph and asking the model only to explain one finding from a bounded witness is none of those things, and this paper reports it costs nothing in detection quality to make the swap: prompt cost grows 1.47× across a 154× growth in project size, 200 of 200 fabricated citations are removed, narrated and computed verdicts agree at $\kappa = 0.95$, and on 270 sprints from 22 unseen projects the same graph warns at the halfway mark with AUC 0.792, rising to 0.874 at three-quarters.

It also reports where the approach does not win. The projection ranks no better than counting unfinished work, and a tuned learned model cannot be statistically separated from a rule with no parameters—after that model's cross-validated F1 of 0.816 collapsed to 0.593 on projects it had not seen.

Three pieces of future work follow directly from the threats above, in order of how much they would change the conclusions: a human study of whether the grounded explanations change what managers do, which is the gap the grounding guarantee cannot close; real-world validation of the five categories currently supported only by synthetic evidence, which needs a data source with intra-sprint dependency structure; and a repeat of the live-model run on a paid key with per-call error logging, which would turn the 54–95% bound into a number.

Acknowledgment

The author thanks the maintainers of the TAWOS dataset for releasing it under terms that permit this kind of secondary analysis.

References

1. V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, "A versatile dataset of agile open source software projects," in *Proc. 19th Int. Conf. Mining Software Repositories (MSR)*, 2022, pp. 420–424, doi: 10.1145/3524842.3528029.
2. A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. 7th Python in Science Conf. (SciPy)*, 2008, pp. 11–15.
3. B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.
4. S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2308.00352.
5. C. Qian *et al.*, "ChatDev: Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Computational Linguistics (ACL)*, 2024, pp. 15174–15186, arXiv:2307.07924.
6. X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2308.03688.
7. S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023, arXiv:2210.03629.
8. S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.
9. M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1135–1144.
10. J. Jiarpakdee, C. Tantithamthavorn, H. K. Dam, and J. Grundy, "An empirical study of model-agnostic techniques for defect prediction models," *IEEE Trans. Softw. Eng.*, vol. 48, no. 1, pp. 166–185, 2022.
11. P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
12. Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.
13. J. Zhou *et al.*, "Graph neural networks: A review of methods and applications," *AI Open*, vol. 1, pp. 57–81, 2020.
14. J. Cohen, "A coefficient of agreement for nominal scales," *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960.
15. J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
16. T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic literature review on fault prediction performance in software engineering," *IEEE Trans. Softw. Eng.*, vol. 38, no. 6, pp. 1276–1304, 2012.
17. M. Jørgensen and M. Shepperd, "A systematic review of software development cost estimation studies," *IEEE Trans. Softw. Eng.*, vol. 33, no. 1, pp. 33–53, 2007.
18. M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, "A deep learning model for estimating story points," *IEEE Trans. Softw. Eng.*, vol. 45, no. 7, pp. 637–656, 2019.
19. V. Tawosi, R. Moussa, and F. Sarro, "Agile effort estimation: Have we solved the problem yet? Insights from a replication study," *IEEE Trans. Softw. Eng.*, 2023, doi: 10.1109/TSE.2022.3228739.
20. Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, arXiv:2308.08155.
21. L. Wang *et al.*, "A survey on large language model based autonomous agents," *Front. Comput. Sci.*, vol. 18, no. 6, art. 186345, 2024, doi: 10.1007/s11704-024-40231-1.
22. R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surv.*, vol. 51, no. 5, pp. 1–42, 2018.
23. A. B. Arrieta *et al.*, "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, 2020.
24. S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 7, pp. 3580–3599, 2024.
25. U. Brandes, "A faster algorithm for betweenness centrality," *J. Math. Sociol.*, vol. 25, no. 2, pp. 163–177, 2001.
26. V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech.: Theory Exp.*, vol. 2008, no. 10, P10008, 2008.